

คู่มือทำ Lab 09–11

Authentication, JWT & Cart

ขั้นตอนละเอียดสำหรับโปรเจกต์ KUSHOP ตั้งแต่สร้างตารางในฐานข้อมูล
เขียน backend และ frontend ไปจนถึงทดสอบระบบสมาชิก การเข้าสู่ระบบ
ด้วย JWT ใน cookie และตะกร้าสินค้าที่เก็บในฐานข้อมูล

โค้ดทั้งหมดในเอกสารนี้ดึงมาจากไฟล์จริงในโปรเจกต์ ณ วันที่สร้างเอกสาร
Node.js ^22.18.0 || >=24.12.0 · Express · PostgreSQL · Vue 3 · Pinia

สารบัญ

ส่วน A สรุปสิ่งที่ทำไปแล้วใน Lab 09–11

ส่วน B สิ่งที่ต้องทำเอง (ทีละขั้น)

ส่วน C ติดตั้งบนเครื่องใหม่หลัง clone / pull จาก GitHub

รายละเอียดแต่ละเรื่อง

1. ภาพรวมระบบ
2. สิ่งที่ต้องเตรียม
3. ฐานข้อมูล: ตาราง members, carts และ cartDtl
4. ตั้งค่าไฟล์ .env
5. Backend สมาชิก: memberController, memberRoute, server.js
6. ทดสอบ Backend ด้วย Postman
7. Frontend: Pinia, authStore, Router
8. Frontend: หน้า Login, Register, PageMember และ MainMenu
9. Lab 11 Backend: ตะกร้าสินค้า (cartController)
10. Lab 11 Frontend: cartStore และหน้าตะกร้า
11. รันและทดสอบทั้งระบบ
12. ปัญหาที่พบบ่อยและวิธีแก้
13. สิ่งที่ต่างจากเอกสารแลป

A สิ่งที่ทำไปแล้วใน Lab 09–11

ส่วนนี้สรุปว่าในโปรเจกต์มีอะไรทำเสร็จแล้วบ้าง **ไม่ต้องเขียนโค้ดเหล่านี้ซ้ำ** โค้ดเต็มอยู่ในหัวข้อ 5, 7–10

A.1 Lab 09 – Authentication

หัวข้อในเอกสารแลป	ทำแล้วที่ไฟล์
<code>postMember</code> : ตรวจสอบข้อมูลครบ, ตรวจสอบอีเมลซ้ำ, hash รหัสผ่านด้วย <code>bcrypt (salt 11)</code> , INSERT ลง <code>members</code>	<code>controllers/memberController.js</code>
<code>loginMember</code> : หาอีเมล แล้วเทียบรหัสด้วย <code>bcrypt.compare</code>	<code>controllers/memberController.js</code>
route <code>POST /members</code> และ <code>POST /members/login</code>	<code>src/router/memberRoute.js</code>
ฟอร์ม Login (ใช้อีเมลเป็น <code>loginName</code>) และฟอร์ม Register ที่ <code>/register</code>	<code>TheLogin.vue</code> , <code>TheRegister.vue</code>

A.2 Lab 10 – Authorization JWT

หัวข้อในเอกสารแลป	ทำแล้วที่ไฟล์
CORS กำหนด origin + <code>credentials: true</code> , เรียก <code>cookieParser()</code> ก่อน router	<code>server.js</code>
<code>SECRET_KEY</code> ใน <code>.env</code>	<code>.env</code>
login สำเร็จสร้าง JWT อายุ 1 ชม. เก็บใน cookie (<code>httpOnly</code> , <code>secure</code> , <code>sameSite: strict</code>), login ผิดลบ cookie	<code>controllers/memberController.js</code>
<code>getMember</code> → <code>GET /members/detail</code> , <code>logoutMember</code> → <code>GET /members/logout</code>	<code>memberController.js</code> , <code>memberRoute.js</code>
<code>axios.defaults.withCredentials = true</code> และติดตั้ง Pinia	<code>src/main.js</code>
หน้า <code>PageMember</code> , เมนูแสดงชื่อสมาชิก + ปุ่มออกจากระบบ, ถ้า login แล้วเข้า <code>/login</code> ให้เปลี่ยนหน้า	<code>PageMember.vue</code> , <code>MainMenu.vue</code> , <code>src/router/index.js</code>

A.3 Lab 11 – Transaction (ตะกร้าสินค้า)

หัวข้อในเอกสารแลป	ฟังก์ชัน / Endpoint	ไฟล์
ตรวจตะกร้าค้าง	<code>chkCart</code> · POST <code>/carts/chkcart</code>	<code>controllers/cartController.js</code>
สร้างตะกร้า (Gen id)	<code>postCart</code> · POST <code>/carts/addcart</code>	<code>controllers/cartController.js</code>
บันทึกสินค้าเข้าตะกร้า	<code>postCartDtl</code> · POST <code>/carts/addcartdtl</code>	<code>controllers/cartController.js</code>
นำสินค้าออก / ลบตะกร้า	<code>delCartDtl</code> , <code>delCart</code> · DELETE <code>/carts/...</code>	<code>controllers/cartController.js</code>
หาค่ารวม	<code>sumCart</code> · GET <code>/carts/sumcart/:id</code>	<code>controllers/cartController.js</code>
แสดงหัว/รายละเอียดตะกร้า	<code>getCart</code> , <code>getCartDtl</code>	<code>controllers/cartController.js</code>
ตะกร้าทั้งหมดของลูกค้า	<code>getCartByCus</code> · POST <code>/carts/getcartbycus</code>	<code>controllers/cartController.js</code>
หน้ารายละเอียดสินค้า + ปุ่มใส่ตะกร้า	route <code>/productshow/:pdId</code>	<code>ProductShow.vue</code>
แสดงตะกร้าบนเมนู	–	<code>CartInfo.vue</code> ใน <code>MainMenu.vue</code>
หน้ารายละเอียดตะกร้า / ตะกร้าทั้งหมด	route <code>/cartshow/:cartId</code> , <code>/cartlist</code>	<code>CartShow.vue</code> , <code>CartList.vue</code>
Store เก็บจำนวนสินค้าในตะกร้า	<code>useCartStore</code>	<code>src/stores/cartStore.js</code>

ทำเพิ่มจากแลป: แก้จำนวนสินค้าในตะกร้า (`setCartDtlQty`) และยืนยันสั่งซื้อ (`cfCart` → ตั้ง `cartCf` เป็น true) ซึ่งแลปมีปุ่มไว้แต่ยังไม่ได้เขียนการทำงาน

ตารางเทียบแบบละเอียดพร้อมเหตุผลอยู่ในหัวข้อ 13

B สิ่งที่ต้องทำเอง (ทีละขั้น)

ยังขาดแค่ฝั่งฐานข้อมูล

โค้ดทั้งหมดพร้อมแล้ว แต่ต้องสร้างตาราง `members`, `carts`, `cartDtl` และตั้งค่าการเชื่อมต่อก่อน ถึงจะสมัครสมาชิก เข้าสู่ระบบ และใช้ตะกร้าได้จริง

ขั้นที่ 1 — เตรียม PostgreSQL (หัวข้อ 2, 3.1)

1. ติดตั้งและเปิด PostgreSQL + pgAdmin 4
2. ตรวจสอบว่ามีฐานข้อมูล `kushop` ที่มีตาราง `products/brands` จากแลปก่อนหน้า

ขั้นที่ 2 — สร้าง user dev (หัวข้อ 3.2, ข้ามได้ถ้าใช้ postgres)

SQL (รันในฐานะ postgres)

sql

```
CREATE ROLE dev WITH LOGIN PASSWORD '1234';
GRANT CONNECT ON DATABASE kushop TO dev;
GRANT USAGE ON SCHEMA public TO dev;
```

ขั้นที่ 3 — สร้างตาราง members (หัวข้อ 3.3–3.4)

SQL (รันใน Query Tool ของ kushop)

sql

```
CREATE TABLE public.members (
    "memEmail" VARCHAR(100) NOT NULL PRIMARY KEY,
    "memName" VARCHAR(100),
    "dutyId" VARCHAR(100) DEFAULT 'member',
    "memHash" VARCHAR(255)
);
GRANT SELECT, INSERT, UPDATE, DELETE ON public.members TO dev;
```

ต้องมีเครื่องหมาย " ครอบชื่อคอลัมน์ ไม่อย่างนั้นจะเจอ `column "memEmail" does not exist`

ขั้นที่ 4 — สร้างตาราง carts และ cartDtl (หัวข้อ 3.6, Lab 11)

ถ้าอาจารย์ให้ไฟล์ backup มา ให้ Restore ตาม Lab 11 หน้า 1 หรือรัน SQL นี้แทนก็ได้

SQL (รันใน Query Tool ของ kushop)

sql

```
CREATE TABLE public.carts (
    "cartId" VARCHAR(100) NOT NULL PRIMARY KEY, -- รหัสตะกร้า YYYYMMDD+ลำดับ
    "cusId" VARCHAR(100), -- อีเมลของสมาชิกเจ้าของตะกร้า
    "cartDate" DATE, -- วันที่สร้างตะกร้า
    "cartCf" BOOLEAN DEFAULT false -- ยืนยันสั่งซื้อแล้วหรือยัง
);

CREATE TABLE public."cartDtl" (
    "cartId" VARCHAR(100) NOT NULL,
    "pdId" VARCHAR(100) NOT NULL,
```

```
"qty"    REAL,
"price"  REAL,
PRIMARY KEY ("cartId", "pdId")
);

GRANT SELECT, INSERT, UPDATE, DELETE ON public.carts, public."cartDtl" TO dev;
```

ขั้นที่ 5 — ตรวจสอบตาราง (หัวข้อ 3.5)

```
SQL                                                                    sql

SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_name IN ('members', 'carts', 'cartDtl')
ORDER BY table_name, ordinal_position;
```

ต้องได้ครบ 4 + 4 + 4 คอลัมน์

ขั้นที่ 6 — เติมค่าเชื่อมต่อใน .env (หัวข้อ 4)

.env มี PORT และ SECRET_KEY อยู่แล้ว ห้ามลบ ให้เพิ่มเฉพาะค่าที่ไม่ตรงกับค่าเริ่มต้น (localhost:5432, kushop, postgres / postgres)

```
.env (เพิ่มต่อท้าย)                                                    env

DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=dev
DB_PASSWORD=1234
```

ขั้นที่ 7 — รันระบบ (หัวข้อ 11.1)

```
2 terminal ในโฟลเดอร์โปรเจกต์                                       bash

npm run server    # terminal 1 → http://localhost:3000
npm run dev       # terminal 2 → http://localhost:5173
```

- ถ้า server ตัวเก่าเปิดอยู่ให้ปิด (Ctrl+C) แล้วเปิดใหม่ เพื่อให้อ่าน .env ใหม่
- เปิดเว็บด้วย http://localhost:5173 เท่านั้น ห้ามใช้ 127.0.0.1 เพราะ cookie จะไม่ถูกส่ง

ขั้นที่ 8 — ทดสอบ (หัวข้อ 6 และ 11.2)

ทดสอบ API ด้วย Postman ตามหัวข้อ 6 แล้วทดสอบบนหน้าเว็บตามรายการในหัวข้อ 11.2

ติดปัญหา?

ดูตารางอาการ สาเหตุ และวิธีแก้ในหัวข้อ 12

C ติดตั้งบนเครื่องใหม่ (หลัง clone / pull)

ใช้ตอนย้ายโปรเจกต์ไปเครื่องอื่น GitHub เก็บเฉพาะโค้ด สิ่งที่ไม่ได้ติดไปด้วย มีดังนี้:

สิ่งที่ไม่อยู่ใน git	เหตุผล	ต้องทำอะไร
node_modules/	ไฟล์ใหญ่ ติดตั้งใหม่ได้จาก package.json	npm install (ขั้นที่ C.3)
.env	มีรหัสผ่าน DB และ SECRET_KEY ห้ามขึ้น GitHub	สร้างใหม่ (ขั้นที่ C.4)
ฐานข้อมูล kushop	อยู่ใน PostgreSQL ของแต่ละเครื่อง ไม่ใช่ไฟล์ในโปรเจกต์	ย้ายหรือสร้างใหม่ (ขั้นที่ C.5)

C.1 ติดตั้งโปรแกรมบนเครื่องใหม่

โปรแกรม	ตรวจสอบด้วยคำสั่ง
Git	git --version
Node.js ^22.18.0 >=24.12.0	node -v
PostgreSQL + pgAdmin 4	เปิด pgAdmin แล้วเชื่อมต่อ server ได้

C.2 เอาโค้ดลงเครื่อง

ครั้งแรก	bash
<pre>git clone https://github.com/Deeptiony/WebT.git cd WebT</pre>	

มีโปรเจกต์อยู่แล้ว	bash
<pre>git checkout main git pull origin main</pre>	

C.3 ติดตั้ง package

Terminal	bash
<pre>npm install</pre>	

ต้องรันใหม่ทุกครั้งที pull แล้ว package.json เปลี่ยน

C.4 สร้างไฟล์ .env

Terminal

bash

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

.env

env

```
PORT=3000

DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=postgres
DB_PASSWORD=รหัสผ่าน-postgres-ของเครื่องนี้

SECRET_KEY=วางค่าที่สุ่มได้ตรงนี้
```

C.5 เตรียมฐานข้อมูลบนเครื่องใหม่

ต้องมีตาราง `products`, `brands`, `pdTypes`, `members`, `carts`, `cartDtl`

เครื่องเดิม: export / เครื่องใหม่: import

bash

```
pg_dump -U postgres -d kushop -f kushop.sql      # เครื่องเดิม
createdb -U postgres kushop                      # เครื่องใหม่
psql -U postgres -d kushop -f kushop.sql         # เครื่องใหม่
```

อย่า commit ไฟล์ `kushop.sql` ขึ้น GitHub เพราะมี hash รหัสผ่านสมาชิกอยู่ ถ้าไม่ย้ายข้อมูล ให้สร้างตารางใหม่ตามส่วน B ขั้นที่ 3-4

C.6 รันและอัปเดตครั้งต่อไป

Terminal

bash

```
npm run server    # terminal 1
npm run dev       # terminal 2

git pull origin main
npm install        # ถ้า package.json เปลี่ยน
# ปิด npm run server (Ctrl+C) แล้วเปิดใหม่ เพื่อให้โค้ด backend ใหม่มีผล
```

C.7 ปัญหาที่เจอบ่อยบนเครื่องใหม่

อาการ	สาเหตุ	วิธีแก้
PowerShell: <code>npm.ps1 cannot be loaded</code>	Windows ปิดการรันสคริปต์	ใช้ Command Prompt หรือรัน <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code>
<code>Cannot find module 'multer'</code> (หรือ package อื่น)	ยังไม่ได้ <code>npm install</code> หลัง pull	รัน <code>npm install</code>
server ไม่ขึ้น: <code>SECRET_KEY is missing</code>	ไม่มีไฟล์ <code>.env</code> หรือไฟล์ผิดที่	ทำขั้นที่ C.4 ไฟล์ต้องอยู่ข้าง <code>server.js</code>
<code>EADDRINUSE :::3000</code>	มีโปรแกรมอื่นใช้ port 3000	ปิดโปรแกรมนั้น (หน้าเว็บเรียก API ที่ port 3000 ตายตัว)
Vite เปิดที่ 5174 แล้วล็อกอินไม่ได้	port 5173 ถูกใช้อยู่ และ CORS อนุญาตแค่ 5173	ปิดโปรแกรมที่ใช้ 5173 หรือ <code>npm run dev -- --port 5173 --strictPort</code>
<code>git push</code> ได้ 403	Windows จำบัญชี GitHub อื่นไว้	Credential Manager → ลบ <code>git:https://github.com</code> แล้ว push ใหม่

1 ภาพรวมระบบ

Lab 09 (Authentication) สมัครสมาชิกและเข้าสู่ระบบ โดยเก็บรหัสผ่านเป็นค่า hash ของ bcrypt ในตาราง `members`

Lab 10 (Authorization JWT) เข้าสู่ระบบสำเร็จ backend สร้าง JWT ส่งกลับใน cookie แบบ `httpOnly` ทุกคำขอถัดไป browser จะแนบ cookie ไปด้วย backend จึงรู้ว่าใครใช้งานอยู่โดยไม่ต้องเก็บสถานะที่ server

Lab 11 (Transaction) ตะกร้าสินค้าเก็บในฐานข้อมูล 2 ตาราง คือ `carts` (หัวตะกร้า) กับ `cartDtl` (รายการสินค้า) ผูกกับสมาชิกที่ล็อกอินอยู่



Endpoint	หน้าที่
POST <code>/members</code> · POST <code>/members/login</code>	สมัครสมาชิก · เข้าสู่ระบบ (ตั้ง cookie)
GET <code>/members/detail</code> · GET <code>/members/logout</code>	ถอด token · ลบ cookie
POST <code>/carts/chkcart</code> · POST <code>/carts/addcart</code>	ตรวจตะกร้าค้าง · สร้างตะกร้า
POST <code>/carts/addcartdtl</code> · PUT <code>/carts/setcartdtlqty</code>	ใส่สินค้า · แก้จำนวน
DELETE <code>/carts/delcartdtl/:id/:pdId</code> · DELETE <code>/carts/delcart/:id</code>	ลบสินค้า · ลบตะกร้า
PUT <code>/carts/cfcart/:id</code>	ยืนยันสั่งซื้อ
GET <code>/carts/sumcart/:id</code>	ยอดรวมจำนวนและเงิน
GET <code>/carts/getcart/:id</code> · GET <code>/carts/getcartdtl/:id</code>	หัวตะกร้า · รายการในตะกร้า
POST <code>/carts/getcartbycus</code>	ตะกร้าทั้งหมดของสมาชิกที่ล็อกอิน

2 สิ่งที่ต้องเตรียม

โปรแกรม	ใช้ทำอะไร
Node.js ^22.18.0 >=24.12.0	รัน backend และ Vite
PostgreSQL + pgAdmin 4	ฐานข้อมูล kushop
Postman (ไม่บังคับ)	ทดสอบ API ก่อนต่อกับหน้าเว็บ
Google Chrome / Edge	ทดสอบหน้าเว็บ และดู cookie ใน DevTools

```
Terminal bash
npm install
```

ถ้าทำเองจากโปรเจกต์เปล่า ให้ติดตั้งทีละตัวตามแลป:

```
Terminal bash
npm install bcryptjs          # hash รหัสผ่าน (Lab 09)
npm install jsonwebtoken      # สร้าง/ตรวจ JWT (Lab 10)
npm install cookie-parser     # อ่าน cookie ผัง backend (Lab 10)
npm install pinia             # state management ผัง Vue (Lab 10-11)
```

ทำไมใช้ bcryptjs ไม่ใช่ bcrypt

โปรเจกต์นี้มี bcryptjs อยู่แล้ว วิธีเรียกใช้ (hash , compare) เหมือน bcrypt ทุกอย่าง และไม่ต้องคอมไพล์ native module บน Windows

3 ฐานข้อมูล

ทำใน pgAdmin 4 ตามเอกสาร Lab 09 หน้า 2 และ Lab 11 หน้า 1–2 หรือรันคำสั่ง SQL ในส่วน B ขั้นที่ 3–4 แทนก็ได้ ผลเหมือนกัน

3.1 ตรวจสอบว่ามีฐานข้อมูล kushop

- 1 เปิด pgAdmin แล้วเชื่อมต่อ server PostgreSQL
- 2 คูณใน **Databases** ว่ามี `kushop` ถ้ายังไม่มี คลิกขวา **Databases** → **Create** → **Database...**
- 3 คลิกขวาที่ `kushop` → **Query Tool** เพื่อเปิดหน้าต่างรัน SQL

3.2 สร้าง login role dev

เอกสารแลบใช้ user `dev` รหัส `1234` ถ้าจะใช้ `postgres` ก็ข้ามได้ (SQL อยู่ในส่วน B ขั้นที่ 2)

3.3 ตาราง members (Lab 09)

Name	Data type	Length	Not NULL?	Primary key?	Default
<code>memEmail</code>	character varying	100	✓	✓	
<code>memName</code>	character varying	100			
<code>dutyId</code>	character varying	100			<code>'member'</code>
<code>memHash</code>	character varying	255			

3.4 ตาราง carts (Lab 11)

Name	Data type	Length	Not NULL?	Primary key?	ความหมาย
<code>cartId</code>	character varying	100	✓	✓	รหัสตะกร้า
<code>cusId</code>	character varying	100			อีเมลสมาชิกเจ้าของตะกร้า
<code>cartDate</code>	date				วันที่สร้างตะกร้า
<code>cartCf</code>	boolean				ยืนยันสั่งซื้อแล้วหรือยัง (default <code>false</code>)

3.5 ตาราง cartDtl (Lab 11)

Name	Data type	Length	Not NULL?	Primary key?	ความหมาย
cartId	character varying	100	✓	✓	รหัสตะกร้า
pdId	character varying	100	✓	✓	รหัสสินค้า
qty	real				จำนวน
price	real				ราคาต่อหน่วยตอนใส่ตะกร้า

3.6 กำหนดสิทธิ์

คลิกขวาที่ตาราง → **Properties** → **Security** แล้วให้สิทธิ์ INSERT, SELECT, UPDATE, DELETE กับ dev หรือรัน:

SQL

sql

```
GRANT SELECT, INSERT, UPDATE, DELETE ON public.members, public.carts, public."cartDtl" TO dev;
```

ต้องมีเครื่องหมาย " รอบชื่อคอลัมน์

PostgreSQL จะแปลงชื่อที่ไม่มี " เป็นตัวพิมพ์เล็กทั้งหมด (cartId → cartid) แต่ controller query ด้วย "cartId" ถ้าสร้างผิดจะเจอ error column "cartId" does not exist

4 ตั้งค่าไฟล์ .env

สร้างไฟล์ `.env` ที่โฟลเดอร์หลักของโปรเจกต์ (ระดับเดียวกับ `server.js`) ไฟล์นี้อยู่ใน `.gitignore` แล้ว จะไม่ถูก commit

.env

env

```
PORT=3000

# การเชื่อมต่อฐานข้อมูล (ชื่อตัวแปรตาม database.js ของโปรเจกต์นี้)
DB_HOST=localhost
DB_PORT=5432
DB_NAME=kushop
DB_USER=dev
DB_PASSWORD=1234

# กุญแจลับสำหรับเซ็น JWT
SECRET_KEY=ใส่ค่าสุ่มยาวๆ-ที่นี่
```

database.js

js

```
import pg from "pg";
import dotenv from "dotenv";

dotenv.config();

const { Pool } = pg;

const database = new Pool({
  host: process.env.DB_HOST || "localhost",
  port: Number(process.env.DB_PORT || 5432),
  database: process.env.DB_NAME || "kushop",
  user: process.env.DB_USER || "postgres",
  password: process.env.DB_PASSWORD || "postgres"
});

export default database;
```

สุ่มค่า SECRET_KEY

bash

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

ห้ามเปิดเผย SECRET_KEY

ใครรู้ค่านี้อาจสร้าง token ปลอมเป็นสมาชิกคนไหนก็ได้ อย่าใช้ค่าง่ายๆ ในงานจริง และอย่า commit ไฟล์ `.env` ถ้า server เปิดโดยไม่มี `SECRET_KEY` จะหยุดทำงานพร้อมข้อความ `SECRET_KEY is missing`

5 Backend สมาชิก

ฟังก์ชัน	แลป	ทำอะไร
postMember	09	ตรวจสอบข้อมูลครบ → ตรวจสอบอีเมลซ้ำ → hash รหัสผ่าน (salt 11) → INSERT
loginMember	09 + 10	หาอีเมล → bcrypt.compare → สำเร็จ: jwt.sign + res.cookie / ล้มเหลว: res.clearCookie
getMember	10	อ่าน req.cookies.token → jwt.verify → ส่งข้อมูลสมาชิก
logoutMember	10	res.clearCookie('token')

controllers/memberController.js

js

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import database from "../database.js";

// httpOnly keeps the token away from page JavaScript (XSS); strict stops cross-site sends.
const cookieOptions = {
  httpOnly: true,
  secure: true,
  sameSite: 'strict'
};

export async function postMember(req, res) {
  console.log(`POST /members is requested.`);
  const bodyData = req.body;
  try {
    if (!bodyData.memEmail || !bodyData.memName || !bodyData.password) {
      return res.json({ message: `ERROR memEmail, memName and password are required.`,
regist: false });
    }
    const chkRow = await database.query({
      text: `SELECT * FROM members WHERE "memEmail" = $1`,
      values: [bodyData.memEmail]
    });
    if (chkRow.rowCount !== 0) {
      return res.json({ message: `ERROR memEmail ${bodyData.memEmail} is exists.`, regist:
false });
    }

    // Store only the bcrypt hash, never the password itself
    const saltround = 11;
    const pwdHash = await bcrypt.hash(bodyData.password, saltround);
    await database.query({
      text: `INSERT INTO "members" ("memEmail", "memName", "memHash")
VALUES ($1, $2, $3)`,
      values: [bodyData.memEmail, bodyData.memName, pwdHash]
    });
    // Echo back only public fields, not the password
    res.json({
```

```

        memEmail: bodyData.memEmail,
        memName: bodyData.memName,
        createDate: new Date(),
        message: "Regist Success",
        regist: true
    });
}
catch (err) {
    return res.json({ message: err.message || `Server error`, regist: false });
}
}

export async function loginMember(req, res) {
    console.log(`POST /members/login is requested.`);
    const bodyData = req.body;
    try {
        if (!bodyData.loginName || !bodyData.password) {
            return res.json({ message: `Login and Password is required`, login: false });
        }
        const result = await database.query({
            text: `SELECT * FROM members WHERE "memEmail" = $1`,
            values: [bodyData.loginName]
        });
        if (result.rowCount == 0) {
            return res.json({ message: `Login Fail`, login: false });
        }

        const loginOK = await bcrypt.compare(bodyData.password, result.rows[0].memHash);
        if (loginOK) {
            // Token payload: identifying but not secret
            const theuser = {
                memEmail: result.rows[0].memEmail,
                memName: result.rows[0].memName,
                dutyId: result.rows[0].dutyId
            };
            const token = jwt.sign(theuser, process.env.SECRET_KEY, { expiresIn: '1h' });
            res.cookie('token', token, { ...cookieOptions, maxAge: 3600000 });
            res.json({ message: `Login Success`, login: true });
        }
        else {
            res.clearCookie('token', cookieOptions);
            res.json({ message: `Login Fail`, login: false });
        }
    }
    catch (err) {
        return res.json({ message: err.message || `Server error`, login: false });
    }
}

export async function getMember(req, res) {
    console.log(`GET /members/detail is requested.`);
    const token = req.cookies.token;
    if (!token)
        return res.json({ message: `No member`, login: false });
    try {
        // verify fails if the token or its signature was tampered with
        const member = jwt.verify(token, process.env.SECRET_KEY);
        return res.json({

```

```

        memEmail: member.memEmail,
        memName: member.memName,
        dutyId: member.dutyId,
        login: true
    });
}
catch (err) {
    console.log(err.message);
    return res.json({ message: `The information was falsified.`, login: false });
}
}

export async function updateMember(req, res) {
    const token = req.cookies.token;
    if (!token) {
        return res.status(401).json({ error: 'profile.notAuthenticated' });
    }

    const { memName, currentPassword, newPassword, confirmPassword } = req.body;
    const nextName = typeof memName === 'string' ? memName.trim() : '';

    if (!nextName) {
        return res.status(400).json({ error: 'profile.nameRequired' });
    }
    if (nextName.length > 100) {
        return res.status(400).json({ error: 'profile.nameTooLong' });
    }
    if (newPassword || currentPassword || confirmPassword) {
        if (!currentPassword || !newPassword || !confirmPassword) {
            return res.status(400).json({ error: 'profile.passwordFieldsRequired' });
        }
        if (newPassword.length < 6) {
            return res.status(400).json({ error: 'profile.passwordTooShort' });
        }
        if (newPassword !== confirmPassword) {
            return res.status(400).json({ error: 'profile.passwordMismatch' });
        }
    }

    try {
        const member = jwt.verify(token, process.env.SECRET_KEY);
        const result = await database.query({
            text: `SELECT "memEmail", "memHash", "dutyId"
                FROM "members" WHERE "memEmail" = $1;`,
            values: [member.memEmail]
        });
        const currentMember = result.rows[0];

        if (!currentMember) {
            return res.status(404).json({ error: 'profile.notFound' });
        }
        if (newPassword && !(await bcrypt.compare(currentPassword, currentMember.memHash))) {
            return res.status(401).json({ error: 'profile.currentPasswordWrong' });
        }

        const nextHash = newPassword
            ? await bcrypt.hash(newPassword, 11)
            : currentMember.memHash;
    }

```

```

    await database.query({
      text: `UPDATE "members"
        SET "memName" = $1, "memHash" = $2
        WHERE "memEmail" = $3;`,
      values: [nextName, nextHash, member.memEmail]
    });

    const nextToken = jwt.sign({
      memEmail: member.memEmail,
      memName: nextName,
      dutyId: currentMember.dutyId
    }, process.env.SECRET_KEY, { expiresIn: '1h' });
    res.cookie('token', nextToken, { ...cookieOptions, maxAge: 3600000 });
    return res.json({
      memEmail: member.memEmail,
      memName: nextName,
      dutyId: currentMember.dutyId,
      login: true
    });
  } catch (err) {
    if (err.name === 'JsonWebTokenError' || err.name === 'TokenExpiredError') {
      return res.status(401).json({ error: 'profile.notAuthenticated' });
    }
    console.error(err);
    return res.status(500).json({ error: 'profile.updateFail' });
  }
}

export async function logoutMember(req, res) {
  console.log(`GET /members/logout is requested.`);
  try {
    res.clearCookie('token', cookieOptions);
    res.json({ message: `Logout Success`, login: false });
  } catch (err) {
    return res.json({ message: err.message });
  }
}

```

src/router/memberRoute.js

js

```

import express from "express";
import * as memberC from "../controllers/memberController.js";

const router = express.Router();

router.get("/members/detail", memberC.getMember);
router.get("/members/logout", memberC.logoutMember);
router.put("/members/profile", memberC.updateMember);
router.post("/members", memberC.postMember);
router.post("/members/login", memberC.loginMember);

export default router;

```

server.js

- **CORS** แบบระบุโดเมน พร้อม `credentials: true` เพื่อให้ browser ส่ง cookie เข้ามา port ได้
- `app.use(cookieParser())` ต้องเรียก ก่อน router ไม่อย่างนั้น `req.cookies` จะว่าง
- เรียกใช้ `memberRoutes` และ `cartRoutes`

server.js

js

```
import express from "express";
import cors from "cors";
import dotenv from "dotenv";
import cookieParser from "cookie-parser";
import productRoutes from "./src/router/productRoute.js";
import authRoutes from "./src/router/authRoute.js";
import databaseRoutes from "./src/router/databaseRoute.js";
import memberRoutes from "./src/router/memberRoute.js";
import cartRoutes from "./src/router/cartRoute.js";

dotenv.config();

if (!process.env.SECRET_KEY) {
  throw new Error("SECRET_KEY is missing. Add it to .env before starting the API.");
}

const app = express();
const PORT = Number(process.env.PORT || 3000);

// Whitelist the frontend and allow cookies to travel with requests
app.use(cors({
  origin: ["http://localhost:5173", "http://127.0.0.1:5173"],
  methods: ["GET", "POST", "PUT", "DELETE"],
  credentials: true
}));
app.use(express.json());
app.use("/img_pd", express.static("img_pd"));
// Must run before the routers so req.cookies is filled in
app.use(cookieParser());
app.use(productRoutes);
app.use(authRoutes);
app.use(databaseRoutes);
app.use(memberRoutes);
app.use(cartRoutes);

app.listen(PORT, () => {
  console.log(`API server is running at http://localhost:${PORT}`);
});
```

6 ทดสอบ Backend ด้วย Postman

เปิด backend ด้วย `npm run server` แล้วตั้ง Body เป็น raw → JSON

Request	Body	ผลที่ควรได้
POST /members	<code>{"memEmail":"test1@gmail.com","memName":"Mr.First","password":"1234"}</code>	Regist Success . ส่งเข้าได้ ERROR memEmail ... is exists.
POST /members/login	<code>{"loginName":"test1@gmail.com","password":"1234"}</code>	Login Success + cookie token
GET /members/detail	-	ข้อมูลสมาชิก + login: true
GET /members/logout	-	เรียก detail เข้าได้ No member
POST /carts/chkcart	<code>{}</code>	cartExist จริง/เท็จ (ไม่มี cookie ได้ Login is required)
POST /carts/addcart	<code>{}</code>	cartOK: true + เลขตะกร้า
POST /carts/addcartdtl	<code>{"cartId":"...", "pdId":"4"}</code>	cartDtlOK: true
GET /carts/sumcart/:id	-	<code>{ id, qty, money }</code>

ใน pgAdmin คอลัมน์ `memHash` ต้องขึ้นต้นด้วย `$2` (เป็น hash ไม่ใช่รหัสผ่านจริง) และแถวใน `cartDtl` ต้องมีราคาตรงกับตาราง `products`

กรณี	ผลลัพธ์
ไม่มี cookie	<code>{"message":"No member","login":false}</code> / Login is required
แก้ค่า token แม้แต่ตัวเดียว	The information was falsified.
ขอดูตะกร้าของคนอื่น	ได้ผลลัพธ์ว่าง ([])
แก้ตะกร้าที่ยืนยันสั่งซื้อแล้ว	This cart cannot be changed

7 Frontend: Pinia, authStore, Router

7.1 src/main.js

ติดตั้ง Pinia และตั้ง `axios.defaults.withCredentials = true` **ครั้งเดียวที่นี่** ถ้าไม่ตั้ง browser จะไม่ส่ง/รับ cookie ข้าม port

src/main.js

js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import axios from 'axios'
import App from './App.vue'
import router from './router'
import './assets/main.css'
import './assets/skylearn.css'
import './stores/theme.js'

// Send the auth cookie with every API request
axios.defaults.withCredentials = true

const app = createApp(App)
app.use(createPinia())
app.use(router)
app.mount('#app')
```

7.2 src/stores/authStore.js

src/stores/authStore.js

js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'

export const useAuthStore = defineStore('auth', () => {
  const isLogin = ref(false)
  const member = ref(null)
  const checked = ref(false)

  const login = () => {
    isLogin.value = true
  }

  const logout = () => {
    isLogin.value = false
    member.value = null
  }

  // Ask the API to decode the httpOnly token cookie
  const getMember = async () => {
    try {
      const res = await axios.get('http://localhost:3000/members/detail')
      isLogin.value = Boolean(res.data.login)
      member.value = res.data.login ? res.data : null
    }
  }
})
```

```

    } catch (err) {
      console.log(err.message)
      logout()
    }
    checked.value = true
  }

  // Clears the httpOnly cookie on the server, then the local state
  const memLogout = async () => {
    try {
      await axios.get('http://localhost:3000/members/logout')
    } catch (err) {
      console.log(err.message)
    }
    logout()
  }

  const updateProfile = async (profile) => {
    const res = await axios.put('http://localhost:3000/members/profile', profile)
    member.value = res.data
    isLogin.value = true
    return res.data
  }

  // Dev-only: pretend to be signed in when no API/DB is running
  const fakeLogin = () => {
    member.value = { memEmail: 'dev@localhost', memName: 'Dev', dutyId: 'member', login: true }

    isLogin.value = true
    checked.value = true
  }

  return { isLogin, member, checked, login, logout, getMember, memLogout, updateProfile,
    fakeLogin }
})

```

7.3 src/router/index.js

- `meta.requiresAuth` = ต้องล็อกอินก่อน, `meta.guestOnly` = เฉพาะคนที่ยังไม่ล็อกอิน
- `beforeEach` เช็ก cookie ผ่าน `authStore.getMember()` ครั้งแรกที่เปิดเว็บ แล้วตัดสินใจว่าจะให้ไปหน้าไหน
- route ของ Lab 11: `/productshow/:pdId`, `/cartshow/:cartId`, `/cartlist`

src/router/index.js

js

```

import { createRouter, createWebHistory } from 'vue-router';
import TheHome from '../components/TheHome.vue';
import TheProduct from '../components/PageProduct.vue';
import TheLogin from '../components/TheLogin.vue';
import TheRegister from '../components/TheRegister.vue';
import PageMember from '../components/PageMember.vue';
import ProductManage from '../components/ProductManage.vue';
import DatabaseOverview from '../components/DatabaseOverview.vue';
import ProductShow from '../components/ProductShow.vue';
import CartShow from '../components/CartShow.vue';
import CartList from '../components/CartList.vue';
import { useAuthStore } from '../stores/authStore.js';

```

```

const routes = [
  {
    path: '/',
    name: 'Home',
    component: TheHome,
    meta: { requiresAuth: true },
  },
  {
    path: '/product',
    name: 'Product',
    component: TheProduct,
    meta: { requiresAuth: true },
  },
  {
    path: '/manage',
    name: 'ProductManage',
    component: ProductManage,
    meta: { requiresAuth: true },
  },
  {
    path: '/pagemember',
    name: 'PageMember',
    component: PageMember,
    meta: { requiresAuth: true },
  },
  {
    path: '/profile',
    redirect: '/pagemember',
  },
  {
    path: '/database',
    name: 'DatabaseOverview',
    component: DatabaseOverview,
    meta: { requiresAuth: true },
  },
  {
    // The route carries the product id, so the path must name it too
    path: '/productshow/:pdId',
    name: 'ProductShow',
    component: ProductShow,
    meta: { requiresAuth: true },
  },
  {
    path: '/cartshow/:cartId',
    name: 'CartShow',
    component: CartShow,
    meta: { requiresAuth: true },
  },
  {
    path: '/cartlist',
    name: 'CartList',
    component: CartList,
    meta: { requiresAuth: true },
  },
  {
    path: '/cart',
    redirect: '/cartlist',
  },

```

```

    },
    {
      path: '/login',
      name: 'Login',
      component: TheLogin,
      meta: { guestOnly: true },
    },
    {
      path: '/register',
      name: 'Register',
      component: TheRegister,
      meta: { guestOnly: true },
    },
  ],
];

const router = createRouter({
  history: createWebHistory(),
  routes,
});

router.beforeEach(async (to) => {
  const authStore = useAuthStore();
  // Check the token cookie once per page load; login/logout keep the store in sync after that
  if (!authStore.checked) await authStore.getMember();

  if (to.meta.requiresAuth && !authStore.isLogin) {
    return { name: 'Login' };
  }

  // Already signed in: skip the login/register pages
  if (to.meta.guestOnly && authStore.isLogin) {
    return { name: 'PageMember' };
  }
});

export default router;

```

8 Frontend: หน้าสมาชิก

8.1 TheLogin.vue

src/components/TheLogin.vue

vue

```
<template>
  <AuthLayout :heading="t('auth.welcome')" :highlight="t('auth.back')">
    <form @submit.prevent="handleSubmit">
      <h2>{{ t('auth.loginTitle') }}</h2>
      <label>{{ t('auth.email') }}<input v-model.trim="loginName" type="email" required
autofocus autocomplete="username"></label>
      <label>{{ t('auth.password') }}<input v-model="password" type="password" required
autocomplete="current-password"></label>
      <button class="auth-submit sk-btn sk-btn-big sk-btn-block" type="submit"
:disabled="busy">{{ t('auth.submitLogin') }} <span>></span></button>
    </form>

    <p v-if="message" class="auth-message sk-msg" role="status" :class="{ error: !login }">{{
t(message) }}</p>
    <router-link to="/register" class="auth-switch-link">{{ t('auth.toRegister') }}</router-
link>
    <DevBypass v-if="DevBypass" />
  </AuthLayout>
</template>

<script setup>
import { defineAsyncComponent, ref } from 'vue'
import { useRouter } from 'vue-router'
import axios from 'axios'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'
import AuthLayout from './AuthLayout.vue'

const DevBypass = import.meta.env.DEV ? defineAsyncComponent(() => import('../DevBypass.vue')) :
null

const authStore = useAuthStore()
const router = useRouter()
const loginName = ref('')
const password = ref('')
const login = ref(false)
const message = ref('')
const busy = ref(false)

const handleSubmit = async () => {
  busy.value = true
  try {
    const response = await axios.post('http://localhost:3000/members/login', {
      loginName: loginName.value,
      password: password.value
    })
    login.value = response.data.login
    message.value = response.data.message
    if (login.value) {
      authStore.login()
    }
  }
}
```

```

        await authStore.getMember()
        router.push('/pagemember')
    }
} catch (err) {
    console.log(err)
    login.value = false
    message.value = 'auth.loginFail'
} finally {
    busy.value = false
}
}
}
</script>

```

8.2 TheRegister.vue

src/components/TheRegister.vue

vue

```

<template>
  <AuthLayout :heading="t('register.heading')" :highlight="t('register.highlight')">
    <form @submit.prevent="handleSubmit">
      <h2>{{ t('register.title') }}</h2>
      <label>{{ t('auth.email') }}<input v-model.trim="memEmail" type="email" required
autofocus autocomplete="email"></label>
      <label>{{ t('register.name') }}<input v-model.trim="memName" type="text" required
autocomplete="name"></label>
      <label>{{ t('auth.password') }}<input v-model="password" type="password" required
autocomplete="new-password"></label>
      <button class="auth-submit sk-btn sk-btn-big sk-btn-block" type="submit"
:disabled="busy">{{ t('register.submit') }} <span></span></button>
    </form>

    <p v-if="message" class="auth-message sk-msg" role="status" :class="{ error: !regist }">
{{ t(message) }}</p>
    <router-link to="/login" class="auth-switch-link">{{ t('register.toLogin') }}</router-
link>
  </AuthLayout>
</template>

<script setup>
import { ref } from 'vue'
import axios from 'axios'
import { t } from '../i18n.js'
import AuthLayout from './AuthLayout.vue'

const memEmail = ref('')
const memName = ref('')
const password = ref('')
const regist = ref(false)
const message = ref('')
const busy = ref(false)

const handleSubmit = async () => {
  busy.value = true
  try {
    const response = await axios.post('http://localhost:3000/members', {
      memEmail: memEmail.value,
      memName: memName.value,

```

```

        password: password.value
      })
      regist.value = response.data.regist
      message.value = response.data.message
      if (regist.value) password.value = ''
    } catch (err) {
      console.log(err)
      regist.value = false
      message.value = 'register.fail'
    } finally {
      busy.value = false
    }
  }
}
</script>

```

8.3 PageMember.vue

src/components/PageMember.vue (ไม่รวม CSS)

vue

```

<template>
  <section class="member-page">
    <div class="member-card sk-panel">
      <div class="member-avatar" aria-hidden="true">{{ initials }}</div>
      <p class="member-kicker sk-kicker">{{ t('profile.kicker') }}</p>
      <h1>{{ member?.memName }}</h1>

      <dl class="member-details">
        <div><dt>{{ t('auth.email') }}</dt><dd>{{ member?.memEmail }}</dd></div>
        <div><dt>{{ t('member.duty') }}</dt><dd><span class="sk-pill">{{ member?.dutyId
|| '-' }}</span></dd></div>
        <div><dt>{{ t('profile.status') }}</dt><dd class="member-online">{{
t('member.signedIn') }}</dd></div>
      </dl>

      <button class="profile-edit-button sk-btn sk-btn-block" type="button"
@click="openEditor">
        {{ t('profile.edit') }}
      </button>
      <form v-if="editing" class="profile-form" @submit.prevent="saveProfile">
        <label>
          {{ t('register.name') }}
          <input v-model="profileForm.memName" type="text" maxLength="100" required
autocomplete="name">
        </label>
        <label>
          {{ t('profile.currentPassword') }}
          <input v-model="profileForm.currentPassword" type="password"
autocomplete="current-password">
        </label>
        <label>
          {{ t('profile.newPassword') }}
          <input v-model="profileForm.newPassword" type="password" minLength="6"
autocomplete="new-password">
        </label>
        <label>
          {{ t('profile.confirmPassword') }}
          <input v-model="profileForm.confirmPassword" type="password" minLength="6"

```

```

autocomplete="new-password">
    </label>
    <p v-if="profileError" class="profile-message profile-error" role="alert">{{
t(profileError) }}</p>
    <p v-if="profileSaved" class="profile-message profile-success" role="status">{{
t('profile.saved') }}</p>
    <div class="profile-form-actions">
        <button class="profile-save sk-btn sk-btn-block" type="submit"
:disabled="saving">
            {{ saving ? t('profile.saving') : t('profile.save') }}
        </button>
        <button class="profile-cancel sk-btn sk-btn-block" type="button"
@click="editing = false">
            {{ t('profile.cancel') }}
        </button>
    </div>
</form>
<router-link to="/product" class="member-shop sk-btn sk-btn-block">{{
t('home.browse') }} →</router-link>
    <button class="member-logout sk-btn sk-btn-coral sk-btn-block" type="button"
@click="logout">{{ t('profile.logout') }}</button>
</div>
</section>
</template>

<script setup>
import { computed, reactive, ref } from 'vue'
import { useRouter } from 'vue-router'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'

const authStore = useAuthStore()
const router = useRouter()
const member = computed(() => authStore.member)
const initials = computed(() => (member.value?.memName || 'KU').slice(0, 2).toUpperCase())
const editing = ref(false)
const saving = ref(false)
const profileError = ref('')
const profileSaved = ref(false)
const profileForm = reactive({ memName: '', currentPassword: '', newPassword: '',
confirmPassword: '' })

const openEditor = () => {
    profileForm.memName = member.value?.memName || ''
    profileForm.currentPassword = ''
    profileForm.newPassword = ''
    profileForm.confirmPassword = ''
    profileError.value = ''
    profileSaved.value = false
    editing.value = true
}

const saveProfile = async () => {
    profileError.value = ''
    profileSaved.value = false
    saving.value = true
    try {
        await authStore.updateProfile({ ...profileForm })
    }
}

```

```

        profileForm.currentPassword = ''
        profileForm.newPassword = ''
        profileForm.confirmPassword = ''
        profileSaved.value = true
    } catch (error) {
        profileError.value = error.response?.data?.error || 'profile.updateFail'
    } finally {
        saving.value = false
    }
}

const logout = async () => {
    await authStore.memLogout()
    router.push('/login')
}
</script>

```

8.4 MainMenu.vue

src/components/MainMenu.vue – ส่วน script

vue

```

<script setup>
import { computed } from 'vue'
import { useRouter } from 'vue-router'
import { isSky } from '../stores/theme.js'
import { useAuthStore } from '../stores/authStore.js'
import { t } from '../i18n.js'
import SkyIcon from './SkyIcon.vue'
import AppSwitches from './AppSwitches.vue'
import CartInfo from './CartInfo.vue'

const authStore = useAuthStore()
const router = useRouter()
const memName = computed(() => authStore.member?.memName)

const memLogout = async () => {
    await authStore.memLogout()
    router.push('/login')
}

const links = [
    { to: '/', label: 'nav.home', icon: 'home' },
    { to: '/product', label: 'nav.products', icon: 'shop' },
    { to: '/login', label: 'nav.login', icon: 'login', guest: true },
    { to: '/manage', label: 'nav.manage', icon: 'pencil', member: true },
    { to: '/database', label: 'nav.database', icon: 'database', member: true },
    { to: '/cartlist', label: 'nav.orders', icon: 'shop', member: true }
]

const visibleLinks = computed(() => links.filter((link) =>
    authStore.isLogin ? !link.guest : !link.member
))
</script>

```

```
<li v-if="authStore.isLogin" class="nav-item">
  <CartInfo />
</li>
<li v-if="authStore.isLogin" class="nav-item">
  <router-link to="/pagemember" style="text-decoration: none"><div class="nav-link fw-bold">{{
memName }}</div></router-link>
</li>
<li v-if="authStore.isLogin" class="nav-item">
  <a href="#" style="text-decoration: none" @click.prevent="memLogout"><div class="nav-link fw-
bold">{{ t('profile.logout') }}</div></a>
</li>
<li v-if="!authStore.isLogin" class="nav-item">
  <router-link to="/login" style="text-decoration: none"><div class="nav-link">{{ t('nav.login')
}}</div></router-link>
</li>
```

9 Lab 11 Backend: ตะกร้าสินค้า

ทุก endpoint อ่านเจ้าของตะกร้าจาก token ใน cookie ไม่ใช่จาก body และตรวจว่าตะกร้าเป็นของคนที่ยล็อกอินอยู่จริง

ฟังก์ชัน	ทำอะไร
<code>readMember</code> / <code>ownsCart</code> / <code>canEditCart</code>	อ่านสมาชิกจาก token · ตรวจเจ้าของ · ตรวจว่าตะกร้ายังไม่ถูกยืนยัน
<code>chkCart</code>	หาตะกร้าที่ยังไม่ยืนยัน (<code>cartCf != true</code>) ของสมาชิกคนนี้
<code>postCart</code>	สร้างเลขตะกร้า <code>YYYYMMDD</code> + ลำดับ 4 หลัก แล้ว INSERT (ถ้ามีตะกร้าค้างอยู่แล้วคืนใบเดิม)
<code>postCartDtl</code>	มีสินค้าตัวเดียวกันอยู่แล้วให้บวกจำนวน ถ้ายังไม่มีให้ INSERT ราคาได้จากตาราง <code>products</code>
<code>setCartDtlQty</code> / <code>delCartDtl</code> / <code>delCart</code>	แก้จำนวน · ลบสินค้า · ลบตะกร้าทั้งใบ
<code>cfCart</code>	ยืนยันสั่งซื้อ: ตั้ง <code>cartCf</code> เป็น true (ตะกร้าว่างยืนยันไม่ได้)
<code>sumCart</code> / <code>getCart</code> / <code>getCartDtl</code> / <code>getCartByCus</code>	ยอดรวม · หัวตะกร้า · รายการในตะกร้า · ตะกร้าทั้งหมดของสมาชิก

controllers/cartController.js

js

```
import jwt from "jsonwebtoken";
import database from "../database.js";

// The cart belongs to the signed-in member, so the owner comes from the token
// cookie, never from the request body – a body value could name anyone.
function readMember(req) {
  const token = req.cookies?.token;
  if (!token) return null;
  try {
    return jwt.verify(token, process.env.SECRET_KEY);
  } catch (err) {
    console.log(err.message);
    return null;
  }
}

async function ownsCart(memEmail, cartId) {
  const result = await database.query({
    text: `SELECT "cusId" FROM carts WHERE "cartId" = $1`,
    values: [cartId]
  });
  return result.rows[0]?.cusId === memEmail;
}

// A confirmed cart is an order already placed: it must not change any more
async function canEditCart(memEmail, cartId) {
  const result = await database.query({
    text: `SELECT "cusId", "cartCf" FROM carts WHERE "cartId" = $1`,
```

```

        values: [cartId]
    });
    const cart = result.rows[0];
    return Boolean(cart) && cart.cusId === memEmail && cart.cartCf !== true;
}

export async function chkCart(req, res) {
    const member = readMember(req);
    console.log(`POST CART customer ${member?.memEmail} is requested`);
    if (!member) {
        return res.json({ cartExist: false, error: true, errorMessage: "Login is required" });
    }
    try {
        // An open cart is one that has not been confirmed yet
        const result = await database.query({
            text: `SELECT * FROM carts WHERE "cusId" = $1 AND "cartCf" != true`,
            values: [member.memEmail]
        });
        if (result.rows[0] != null) {
            return res.json({ cartExist: true, cartId: result.rows[0].cartId });
        }
        return res.json({ cartExist: false });
    } catch (err) {
        return res.json({ cartExist: false, error: true, errorMessage: err.message || `Server error` });
    }
}

export async function postCart(req, res) {
    const member = readMember(req);
    console.log(`POST /CART is requested`);
    if (!member) {
        return res.json({ cartOK: false, messageAddCart: "Login is required" });
    }
    try {
        // One open cart per member: a double click must not create a second one
        const open = await database.query({
            text: `SELECT "cartId" FROM carts WHERE "cusId" = $1 AND "cartCf" != true`,
            values: [member.memEmail]
        });
        if (open.rows[0]) {
            return res.json({ cartOK: true, messageAddCart: open.rows[0].cartId });
        }

        // Cart id is YYYYMMDD + a running number for that day
        const now = new Date();
        const year = now.getFullYear();
        const month = String(now.getMonth() + 1).padStart(2, "0");
        const day = String(now.getDate()).padStart(2, "0");
        const currentDate = `${year}${month}${day}`;

        let i = 0;
        let theId = "";
        let existsResult = [];
        do {
            i++;
            theId = `${currentDate}${String(i).padStart(4, "0")}`;
            existsResult = await database.query({

```

```

        text: `SELECT EXISTS (SELECT * FROM carts WHERE "cartId" = $1)`,
        values: [theId]
    });
} while (existsResult.rows[0].exists);

await database.query({
    text: `INSERT INTO carts ("cartId", "cusId", "cartDate") VALUES ($1, $2, $3)`,
    values: [theId, member.memEmail, now]
});
return res.json({ cartOK: true, messageAddCart: theId });
} catch (err) {
    return res.json({ cartOK: false, messageAddCart: err.message || `Server error` });
}
}

export async function postCartDtl(req, res) {
    const member = readMember(req);
    console.log(`POST /CARTDETAIL is requested`);
    if (!member) {
        return res.json({ cartDtlOK: false, messageAddCartDtl: "Login is required" });
    }
    try {
        if (req.body.cartId == null || req.body.pdId == null) {
            return res.json({
                cartDtlOK: false,
                messageAddCartDtl: "CartId && ProductID is required"
            });
        }
        if (!await canEditCart(member.memEmail, req.body.cartId)) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "This cart cannot be changed"
        });
        }

        // The price comes from the products table, never from the request:
        // a body value would let the buyer pick their own price.
        const priceResult = await database.query({
            text: `SELECT "pdPrice" FROM products WHERE "pdId" = $1`,
            values: [req.body.pdId]
        });
        if (priceResult.rowCount === 0) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "Product not found" });
        }
        const price = priceResult.rows[0].pdPrice;

        // Same product already in the cart? add to its quantity instead of a second row
        const pdResult = await database.query({
            text: `SELECT * FROM "cartDtl" ctd WHERE ctd."cartId" = $1 AND ctd."pdId" = $2`,
            values: [req.body.cartId, req.body.pdId]
        });
        const qty = Math.max(1, Math.floor(Number(req.body.qty)) || 1);

        if (pdResult.rowCount == 0) {
            await database.query({
                text: `INSERT INTO "cartDtl" ("cartId", "pdId", "qty", "price") VALUES ($1, $2,
$3, $4)`,
                values: [req.body.cartId, req.body.pdId, qty, price]
            });
        } else {

```

```

        await database.query({
            text: `UPDATE "cartDtl" SET "qty" = $1 WHERE "cartId" = $2 AND "pdId" = $3`,
            values: [pdResult.rows[0].qty + qty, req.body.cartId, req.body.pdId]
        });
    }
    return res.json({ cartDtlOK: true, messageAddCartDtl: req.body.cartId });
} catch (err) {
    return res.json({ cartDtlOK: false, messageAddCartDtl: err.message || `Server error` });
}
}

export async function setCartDtlQty(req, res) {
    const member = readMember(req);
    console.log(`PUT /CARTDETAIL is requested`);
    if (!member) {
        return res.json({ cartDtlOK: false, messageAddCartDtl: "Login is required" });
    }
    try {
        const qty = Math.floor(Number(req.body.qty))
        if (req.body.cartId == null || req.body.pdId == null || !(qty > 0)) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "CartId && ProductID && Qty is required" });
        }
        if (!await canEditCart(member.memEmail, req.body.cartId)) {
            return res.json({ cartDtlOK: false, messageAddCartDtl: "This cart cannot be changed" });
        }
        await database.query({
            text: `UPDATE "cartDtl" SET "qty" = $1 WHERE "cartId" = $2 AND "pdId" = $3`,
            values: [qty, req.body.cartId, req.body.pdId]
        });
        return res.json({ cartDtlOK: true, messageAddCartDtl: req.body.cartId });
    } catch (err) {
        return res.json({ cartDtlOK: false, messageAddCartDtl: err.message || `Server error` });
    }
}

export async function delCartDtl(req, res) {
    const member = readMember(req);
    console.log(`DELETE /CARTDETAIL ${req.params.id}/${req.params.pdId} is requested`);
    if (!member) {
        return res.json({ delOK: false, message: "Login is required" });
    }
    try {
        if (!await canEditCart(member.memEmail, req.params.id)) {
            return res.json({ delOK: false, message: "This cart cannot be changed" });
        }
        await database.query({
            text: `DELETE FROM "cartDtl" WHERE "cartId" = $1 AND "pdId" = $2`,
            values: [req.params.id, req.params.pdId]
        });
        return res.json({ delOK: true, message: `Delete detail success` });
    } catch (err) {
        return res.json({ delOK: false, message: err.message || `Server error` });
    }
}

export async function delCart(req, res) {

```

```

const member = readMember(req);
console.log(`DELETE /CART ${req.params.id} is requested`);
if (!member) {
    return res.json({ delOK: false, message: "Login is required" });
}
try {
    if (!await canEditCart(member.memEmail, req.params.id)) {
        return res.json({ delOK: false, message: "This cart cannot be changed" });
    }
    // Details first: they reference the cart
    await database.query({
        text: `DELETE FROM "cartDtl" WHERE "cartId" = $1`,
        values: [req.params.id]
    });
    await database.query({
        text: `DELETE FROM carts WHERE "cartId" = $1`,
        values: [req.params.id]
    });
    return res.json({ delOK: true, message: `Delete cart success` });
} catch (err) {
    return res.json({ delOK: false, message: err.message || `Server error` });
}
}

export async function cfCart(req, res) {
    const member = readMember(req);
    console.log(`PUT /CART CONFIRM ${req.params.id} is requested`);
    if (!member) {
        return res.json({ cfOK: false, message: "Login is required" });
    }
    try {
        if (!await canEditCart(member.memEmail, req.params.id)) {
            return res.json({ cfOK: false, message: "This cart cannot be changed" });
        }
        // An empty cart cannot be ordered
        const countResult = await database.query({
            text: `SELECT COUNT(*) AS rows FROM "cartDtl" WHERE "cartId" = $1`,
            values: [req.params.id]
        });
        if (Number(countResult.rows[0].rows) === 0) {
            return res.json({ cfOK: false, message: `Cart is empty` });
        }
        await database.query({
            text: `UPDATE carts SET "cartCf" = true WHERE "cartId" = $1`,
            values: [req.params.id]
        });
        return res.json({ cfOK: true, message: `Confirm success` });
    } catch (err) {
        return res.json({ cfOK: false, message: err.message || `Server error` });
    }
}

export async function sumCart(req, res) {
    console.log(`GET SumCart ${req.params.id} is requested`);
    const member = readMember(req);
    if (!member) return res.json({ id: req.params.id, qty: 0, money: 0 });
    try {
        if (!await ownsCart(member.memEmail, req.params.id)) {

```

```

        return res.json({ id: req.params.id, qty: 0, money: 0 });
    }
    const result = await database.query({
        text: `SELECT SUM(qty) AS qty, SUM(qty * price) AS money
            FROM "cartDtl" ctd WHERE ctd."cartId" = $1`,
        values: [req.params.id]
    });
    return res.json({
        id: req.params.id,
        qty: result.rows[0].qty,
        money: result.rows[0].money
    });
} catch (err) {
    return res.json({ error: err.message || `Server error` });
}
}

export async function getCart(req, res) {
    console.log(`GET Cart ${req.params.id} is requested`);
    const member = readMember(req);
    if (!member) return res.json([]);
    try {
        const result = await database.query({
            text: `SELECT ct.*, SUM(ctd.qty) AS sqty, SUM(ctd.price * ctd.qty) AS sprice
                FROM carts ct LEFT JOIN "cartDtl" ctd ON ct."cartId" = ctd."cartId"
                WHERE ct."cartId" = $1 AND ct."cusId" = $2
                GROUP BY ct."cartId"`,
            values: [req.params.id, member.memEmail]
        });
        return res.json(result.rows);
    } catch (err) {
        return res.json({ error: err.message || `Server error` });
    }
}

export async function getCartDtl(req, res) {
    console.log(`GET CartDtl ${req.params.id} is requested`);
    const member = readMember(req);
    if (!member) return res.json([]);
    try {
        if (!await ownsCart(member.memEmail, req.params.id)) return res.json([]);
        const result = await database.query({
            text: `SELECT ROW_NUMBER() OVER (ORDER BY ctd."pdId") AS row_number,
                ctd."pdId", pd."pdName", ctd.qty, ctd.price
                FROM "cartDtl" ctd LEFT JOIN "products" pd ON ctd."pdId" = pd."pdId"
                WHERE ctd."cartId" = $1
                ORDER BY ctd."pdId"`,
            values: [req.params.id]
        });
        return res.json(result.rows);
    } catch (err) {
        return res.json({ error: err.message || `Server error` });
    }
}

export async function getCartByCus(req, res) {
    console.log(`POST Cart By Customer is requested`);
    const member = readMember(req);

```

```

if (!member) return res.json([]);
try {
  const result = await database.query({
    text: `SELECT ROW_NUMBER() OVER (ORDER BY ct."cartId" DESC) AS row_number,
          ct.*, SUM(ctd.qty) AS sqty, SUM(ctd.price * ctd.qty) AS sprice
          FROM carts ct LEFT JOIN "cartDtl" ctd ON ct."cartId" = ctd."cartId"
          WHERE ct."cusId" = $1
          GROUP BY ct."cartId"
          ORDER BY ct."cartId" DESC`,
    values: [member.memEmail]
  });
  return res.json(result.rows);
} catch (err) {
  return res.json({ error: err.message || `Server error` });
}
}

```

src/router/cartRoute.js

js

```

import express from "express";
import * as cartC from "../../controllers/cartController.js";

const router = express.Router();

router.post("/carts/chkcart", cartC.chkCart);
router.post("/carts/addcart", cartC.postCart);
router.post("/carts/addcartdtl", cartC.postCartDtl);
router.put("/carts/setcartdtlqty", cartC.setCartDtlQty);
router.get("/carts/sumcart/:id", cartC.sumCart);
router.get("/carts/getcart/:id", cartC.getCart);
router.get("/carts/getcartdtl/:id", cartC.getCartDtl);
router.post("/carts/getcartbycus", cartC.getCartByCus);
router.put("/carts/cfcart/:id", cartC.cfCart);
router.delete("/carts/delcartdtl/:id/:pdId", cartC.delCartDtl);
router.delete("/carts/delcart/:id", cartC.delCart);

export default router;

```

10 Lab 11 Frontend: ตะกร้าสินค้า

10.1 src/stores/cartStore.js

Store กลางของตะกร้า เก็บเลขตะกร้า จำนวนชิ้น และยอดเงิน ให้ทุก component ใช้ร่วมกัน

src/stores/cartStore.js

js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'

const API = 'http://localhost:3000'

export const useCartStore = defineStore('cart', () => {
  const cartId = ref(null)
  const theQty = ref(0)
  const money = ref(0)

  // Which cart is still open for this member (not confirmed yet)
  const chkCart = async () => {
    const res = await axios.post(`${API}/carts/chkcart`, {})
    cartId.value = res.data.cartExist ? res.data.cartId : null
    return cartId.value
  }

  const addCart = async () => {
    const res = await axios.post(`${API}/carts/addcart`, {})
    if (!res.data.cartOK) throw new Error(res.data.messageAddCart)
    cartId.value = res.data.messageAddCart
    return cartId.value
  }

  const sumCart = async (id = cartId.value) => {
    if (!id) {
      theQty.value = 0
      money.value = 0
      return
    }
    const res = await axios.get(`${API}/carts/sumcart/${id}`)
    theQty.value = Number(res.data.qty) || 0
    money.value = Number(res.data.money) || 0
  }

  // Read the open cart and its totals; used on load and after every change
  const refresh = async () => {
    try {
      await chkCart()
      await sumCart()
    } catch (err) {
      console.log(err.message)
      reset()
    }
  }

  // Creates the cart on first use, then adds the product to it
```

```

const addProduct = async (product, qty = 1) => {
  if (!cartId.value) await chkCart()
  if (!cartId.value) await addCart()

  const res = await axios.post(`${API}/carts/addcartdtl`, {
    cartId: cartId.value,
    pdId: product.pdId,
    pdPrice: product.pdPrice,
    qty
  })
  if (!res.data.cartDtlOK) throw new Error(res.data.messageAddCartDtl)
  await sumCart()
}

const setQty = async (id, pdId, qty) => {
  const res = await axios.put(`${API}/carts/setcartdtlqty`, { cartId: id, pdId, qty })
  if (!res.data.cartDtlOK) throw new Error(res.data.messageAddCartDtl)
  await sumCart()
}

const removeProduct = async (id, pdId) => {
  const res = await axios.delete(`${API}/carts/delcartdtl/${id}/${pdId}`)
  if (!res.data.delOK) throw new Error(res.data.message)
  await refresh()
}

const deleteCart = async (id) => {
  const res = await axios.delete(`${API}/carts/delcart/${id}`)
  if (!res.data.delOK) throw new Error(res.data.message)
  await refresh()
}

// Confirming closes the cart, so the next add starts a new one
const confirmCart = async (id) => {
  const res = await axios.put(`${API}/carts/cfcart/${id}`)
  if (!res.data.cfOK) throw new Error(res.data.message)
  await refresh()
}

const reset = () => {
  cartId.value = null
  theQty.value = 0
  money.value = 0
}

return { cartId, theQty, money, chkCart, addCart, sumCart, refresh, addProduct, setQty,
  removeProduct, deleteCart, confirmCart, reset }
})

```

10.2 ProductShow.vue (หน้ารายละเอียดสินค้า)

อ่าน `route.params.pdId` แล้วเรียก `GET /products/:id` ปุ่มใส่ตะกร้าจะตรวจการล็อกอินก่อน แล้วเรียก `cartStore.addProduct()` ซึ่งจะตรวจตะกร้าค้าง สร้างตะกร้าถ้ายังไม่มี และบันทึกสินค้าให้ตามลำดับ

```

<template>
  <section class="detail-page sk-page">
    <router-link to="/product" class="detail-back">{{ t('detail.back') }}</router-link>

    <p v-if="loading" class="detail-state" aria-busy="true">{{ t('db.loading') }}</p>
    <div v-else-if="!product" class="detail-state sk-empty">
      <h2>{{ t('detail.notFound') }}</h2>
      <router-link to="/product" class="sk-btn">{{ t('shop.showAll') }}</router-link>
    </div>

    <article v-else class="detail-layout">
      <div class="detail-image sk-card">
        
      </div>

      <div class="detail-info">
        <p class="sk-kicker">{{ t('detail.kicker') }}</p>
        <h1>{{ product.pdName }}</h1>
        <p class="detail-price sk-mono">${{ Number(product.pdPrice).toFixed(2) }}</p>

        <dl class="detail-table">
          <div><dt>{{ t('detail.id') }}</dt><dd class="sk-mono">{{ product.pdId }}</dd>
        </div>

          <div><dt>{{ t('detail.brand') }}</dt><dd>{{ product.brand?.brandName ||
t('product.noBrand') }}</dd></div>
          <div v-if="product.pdt?.pdTypeName"><dt>{{ t('detail.type') }}</dt><dd>{{
product.pdt.pdTypeName }}</dd></div>
          <div><dt>{{ t('detail.remark') }}</dt><dd>{{ product.pdRemark || '-' }}</dd>
        </div>

        </dl>

        <div class="detail-actions">
          <button class="sk-btn sk-btn-big" type="button" :disabled="busy"
@click="addToCart">
            <SkyIcon :name="added ? 'check' : 'cart'" :size="22" />
            {{ t(added ? 'product.added' : 'product.add') }}
          </button>
          <router-link to="/manage" class="sk-btn sk-btn-big sk-btn-ghost">
            <SkyIcon name="pencil" :size="22" /> {{ t('manage.edit') }}
          </router-link>
        </div>

        <p v-if="message" class="sk-msg error" role="status">{{ t(message) }}</p>
        <p v-else-if="added" class="sk-msg" role="status">{{ t('product.addedLive', {
name: product.pdName }) }}</p>
      </div>
    </article>
  </section>
</template>

<script setup>
import { computed, onMounted, ref } from 'vue'
import { useRoute } from 'vue-router'
import axios from 'axios'
import { useCartStore } from '../stores/cartStore.js'
import { useAuthStore } from '../stores/authStore.js'

```

```

import { t } from '../i18n.js'
import SkyIcon from './SkyIcon.vue'

const route = useRoute()
const cartStore = useCartStore()
const authStore = useAuthStore()

const product = ref(null)
const loading = ref(true)
const busy = ref(false)
const added = ref(false)
const message = ref('')

const imageSrc = computed(() => product.value?.logosrc
  ? `http://localhost:3000${product.value.logosrc}`
  : `http://localhost:3000/products/${product.value?.pdId}/image`)

onMounted(async () => {
  try {
    // Backend answers with an array of matching rows
    const res = await axios.get(`http://localhost:3000/products/${route.params.pdId}`)
    product.value = res.data[0] || null
  } catch (err) {
    console.log(err.message)
  } finally {
    loading.value = false
  }
})

const addToCart = async () => {
  if (!authStore.isLogin) {
    message.value = 'product.loginFirst'
    added.value = false
    return
  }
  busy.value = true
  try {
    await cartStore.addProduct(product.value)
    added.value = true
    message.value = ''
  } catch (err) {
    console.log(err.message)
    added.value = false
    message.value = 'product.addFail'
  } finally {
    busy.value = false
  }
}
</script>

```

10.3 CartInfo.vue (ปุ่มตะกร้าบนเมนู)

src/components/CartInfo.vue (ไม่รวม CSS)

vue

```

<template>
  <router-link :to="target" class="cart-info" :class="isSky ? 'sky-cart' : 'nav-link'"
    :aria-label="t('nav.cartAria', { n: cartStore.theQty })">

```

```

        <SkyIcon v-if="isSky" name="cart" :size="24" />
        <span class="cart-info-label">{{ t('nav.cart') }}</span>
        <span class="cart-info-badge sk-mono">{{ cartStore.theQty }}</span>
        <span v-if="cartStore.money > 0" class="cart-info-money sk-mono">${{
cartStore.money.toFixed(2) }}</span>
    </router-link>
</template>

<script setup>
import { computed, onMounted, watch } from 'vue'
import { useCartStore } from '../stores/cartStore.js'
import { useAuthStore } from '../stores/authStore.js'
import { isSky } from '../stores/theme.js'
import { t } from '../i18n.js'
import SkyIcon from './SkyIcon.vue'

const cartStore = useCartStore()
const authStore = useAuthStore()

// No open cart yet? send them to the order history instead of a dead link
const target = computed(() => cartStore.cartId ? `/cartshow/${cartStore.cartId}` : '/cartlist')

// Re-read the cart after a sign-in or sign-out
watch(() => authStore.isLogin, (isLogin) => {
    if (isLogin) cartStore.refresh()
    else cartStore.reset()
})

onMounted(() => {
    if (authStore.isLogin) cartStore.refresh()
})
</script>

```

10.4 CartShow.vue (รายละเอียดตะกร้า)

อ่านหัวตะกร้ากับรายการสินค้าพร้อมกัน ถ้าตะกร้าไม่ใช่ของผู้ใช้คนนี้ API จะคืนค่าว่างและหน้าจะขึ้นว่าไม่มีสิทธิ์ดู ตะกร้าที่ยืนยันแล้วจะซ่อนปุ่มแก้ไขทั้งหมด

src/components/CartShow.vue – ส่วน script

vue

```

<script setup>
import { computed, onMounted, ref, watch } from 'vue'
import { useRoute, useRouter } from 'vue-router'
import axios from 'axios'
import { useCartStore } from '../stores/cartStore.js'
import { isSky } from '../stores/theme.js'
import { t } from '../i18n.js'
import SkyConfetti from './SkyConfetti.vue'

const route = useRoute()
const router = useRouter()
const cartStore = useCartStore()

const cart = ref(null)
const items = ref([])
const loading = ref(true)
const busy = ref(false)

```

```

const done = ref(false)
const message = ref('')
const celebrate = ref(false)

const totalQty = computed(() => items.value.reduce((sum, item) => sum + Number(item.qty), 0))
const totalMoney = computed(() => items.value.reduce((sum, item) => sum + Number(item.price) *
Number(item.qty), 0))

const formattedDate = (value) => {
  if (!value) return '-'
  const date = new Date(value)
  const month = String(date.getMonth() + 1).padStart(2, '0')
  const day = String(date.getDate()).padStart(2, '0')
  return `${date.getFullYear()}-${month}-${day}`
}

const load = async () => {
  loading.value = true
  try {
    const id = route.params.cartId
    // The API only answers with carts that belong to the signed-in member
    const [master, detail] = await Promise.all([
      axios.get(`http://localhost:3000/carts/getcart/${id}`),
      axios.get(`http://localhost:3000/carts/getcartdtl/${id}`)
    ])
    cart.value = Array.isArray(master.data) ? master.data[0] || null : null
    items.value = Array.isArray(detail.data) ? detail.data : []
  } catch (err) {
    console.log(err.message)
    cart.value = null
  } finally {
    loading.value = false
  }
}

const run = async (action) => {
  busy.value = true
  message.value = ''
  try {
    await action()
    return true
  } catch (err) {
    console.log(err.message)
    done.value = false
    message.value = 'cartdb.actionFail'
    return false
  } finally {
    busy.value = false
  }
}

const setQty = (item, qty) => run(async () => {
  await cartStore.setQty(cart.value.cartId, item.pdId, qty)
  await load()
})

const removeItem = (item) => run(async () => {
  await cartStore.removeProduct(cart.value.cartId, item.pdId)

```

```

    await load()
  })

const removeCart = async () => {
  if (!window.confirm(t('cartdb.confirmDelete'))) return
  const ok = await run(() => cartStore.deleteCart(cart.value.cartId))
  if (ok) router.push('/cartList')
}

const confirmOrder = async () => {
  if (!window.confirm(t('cartdb.confirmOrder'))) return
  const ok = await run(() => cartStore.confirmCart(cart.value.cartId))
  if (!ok) return
  done.value = true
  message.value = 'cartdb.done'
  celebrate.value = isSky.value
  await load()
}

watch(() => route.params.cartId, load)
onMounted(load)
</script>

```

10.5 CartList.vue (ตะกร้าทั้งหมดที่เคยสั่ง)

src/components/CartList.vue (ไม่รวม CSS)

vue

```

<template>
  <section class="orders-page sk-page">
    <header class="orders-head sk-head">
      <p class="orders-kicker sk-kicker">{{ t('orders.kicker') }}</p>
      <h1>{{ t('orders.title') }}</h1>
      <p>{{ t('orders.lead') }}</p>
    </header>

    <p v-if="loading" class="orders-state" aria-busy="true">{{ t('db.loading') }}</p>

    <div v-else-if="carts.length" class="orders-table sk-card">
      <table>
        <thead>
          <tr>
            <th>{{ t('orders.no') }}</th>
            <th>{{ t('orders.cartId') }}</th>
            <th>{{ t('orders.date') }}</th>
            <th class="align-center">{{ t('cartdb.qty') }}</th>
            <th class="align-right">{{ t('cartdb.amount') }}</th>
            <th class="align-center">{{ t('orders.status') }}</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="cart in carts" :key="cart.cartId">
            <td class="sk-mono">{{ cart.row_number }}</td>
            <td>
              <router-link :to="`/cartshow/${cart.cartId}`" class="sk-mono order-link">{{ cart.cartId }}</router-link>
            </td>
            <td>{{ formattedDate(cart.cartDate) }}</td>

```

```

        <td class="align-center sk-mono">{{ cart.sqty ?? 0 }}</td>
        <td class="align-right sk-mono">${{ Number(cart.sprice ?? 0).toFixed(2)
    }}</td>

    <td class="align-center">
        <span class="status" :class="cart.cartCf ? 'done' : 'open'">
            {{ cart.cartCf ? t('cartdb.confirmed') : t('cartdb.open') }}
        </span>
    </td>
</tr>
</tbody>
</table>
</div>

<div v-else class="sk-empty">
    <span class="sk-empty-icon">o</span>
    <h2>{{ t('orders.empty') }}</h2>
    <p>{{ t('orders.emptyText') }}</p>
    <router-link to="/product" class="sk-btn">{{ t('cart.browse') }}</router-link>
</div>
</section>
</template>

<script setup>
import { onMounted, ref } from 'vue'
import axios from 'axios'
import { t } from '../i18n.js'

const carts = ref([])
const loading = ref(true)

const formattedDate = (value) => {
    if (!value) return '-'
    const date = new Date(value)
    const month = String(date.getMonth() + 1).padStart(2, '0')
    const day = String(date.getDate()).padStart(2, '0')
    return `${date.getFullYear()}-${month}-${day}`
}

onMounted(async () => {
    try {
        // POST so the member comes from the token cookie, not the URL
        const res = await axios.post('http://localhost:3000/carts/getcartbycus', {})
        carts.value = Array.isArray(res.data) ? res.data : []
    } catch (err) {
        console.log(err.message)
    } finally {
        loading.value = false
    }
})
</script>

```

11 รันและทดสอบทั้งระบบ

11.1 เปิดโปรแกรม (2 terminal)

Terminal 1 – Backend	bash
<pre>npm run server # API server is running at http://localhost:3000</pre>	
Terminal 2 – Frontend	bash
<pre>npm run dev # Local: http://localhost:5173/</pre>	

เปิดเว็บด้วย `http://localhost:5173` เท่านั้น

cookie ตั้งค่า `sameSite: strict` ถ้าเปิดด้วย `127.0.0.1:5173` browser จะมองว่า `localhost:3000` เป็นคนละเว็บ แล้วไม่ส่ง cookie ทำให้ล็อกอินแล้วหลุดทันที

11.2 รายการทดสอบ

ข้อความในตารางเป็นข้อความเมื่อเลือกภาษา **ไทย** (ปุ่ม EN / ไทย มุมบนขวา) ค่าเริ่มต้นของเว็บเป็นภาษาอังกฤษ

✓	ทำอะไร	ผลที่ควรได้
☐	เปิด <code>http://localhost:5173/</code> ตอนยังไม่ล็อกอิน	ถูกพาไป <code>/login</code>
☐	กด "ยังไม่มีบัญชี? ลงทะเบียนใหม่" แล้วสมัคร	ข้อความ "ลงทะเบียนสำเร็จ" และมีแถวใหม่ในตาราง <code>members</code>
☐	ล็อกอินด้วยรหัสผิด / รหัสถูก	"อีเมลหรือรหัสผ่านไม่ถูกต้อง" / ไปหน้า <code>/pagemember</code>
☐	F12 → Application → Cookies	มี <code>token</code> ตี๊ก <code>HttpOnly</code> และ <code>Secure</code>
☐	หน้าสินค้า กด "ดูรายละเอียด"	ไปหน้า <code>/productshow/:pdId</code> เห็นราคายี่ห้อ รายละเอียด
☐	กด "เพิ่มลงตะกร้า"	ปุ่มตะกร้าบนเมนูขึ้นจำนวนและยอดเงิน · ตาราง <code>carts</code> มีแถวใหม่ และ <code>cartDtl</code> มีสินค้า
☐	กดปุ่มตะกร้าบนเมนู	ไปหน้า <code>/cartshow/:cartId</code> เห็นรายการสินค้าและยอดรวม
☐	กดปุ่ม + / - และปุ่ม ×	จำนวนและยอดรวมเปลี่ยนตาม · ลบสินค้าออกจากตะกร้าได้
☐	ใส่สินค้าตัวเดิมซ้ำ	จำนวนบวกเพิ่มในแถวเดิม ไม่เกิดแถวใหม่
☐	กด "ยืนยันสั่งซื้อสินค้า"	ตะกร้าเปลี่ยนสถานะเป็น "สั่งซื้อแล้ว" ปุ่มแก้ไขหายไป และ <code>cartCf</code> ในฐานข้อมูลเป็น <code>true</code>
☐	ใส่สินค้าใหม่หลังยืนยัน	ระบบสร้างตะกร้าใบใหม่ให้

☐	เข้าเมนู "คำสั่งซื้อ"	เห็นตะกร้าทุกใบของตัวเอง เรียงจากล่าสุด
☐	กดออกจากระบบ	ไปหน้า /login , cookie token หายไป

12 ปัญหาที่พบบ่อยและวิธีแก้

อาการ / ข้อความ	สาเหตุ	วิธีแก้
หน้าเว็บขึ้น "เชื่อมต่อเซิร์ฟเวอร์ไม่สำเร็จ" / Server error	PostgreSQL ไม่ได้เปิด หรือค่าใน .env ผิด	เปิด service PostgreSQL ตรวจสอบ DB_* แล้วรีสตาร์ท <code>npm run server</code>
<code>relation "members" does not exist</code> / <code>"carts"</code> / <code>"cartDtl"</code>	ยังไม่ได้สร้างตาราง หรือสร้างผิดฐานข้อมูล	ทำส่วน B ขั้นที่ 3-4 ในฐานข้อมูลที่ตรงกับ DB_NAME
<code>column "cartId" does not exist</code>	สร้างคอลัมน์โดยไม่มี " " จึงกลายเป็นตัวพิมพ์เล็ก	ลบตารางแล้วสร้างใหม่ตาม SQL ในส่วน B
<code>permission denied for table carts</code>	user dev ไม่มีสิทธิ์	รัน GRANT ในหัวข้อ 3.6
กดใส่ตะกร้าแล้วขึ้นว่าต้องเข้าสู่ระบบ	cookie หมดอายุ (1 ชม.) หรือถูกลบ	ล็อกอินใหม่
<code>This cart cannot be changed</code>	ตะกร้าใบนั้นยืนยันสั่งซื้อไปแล้ว	ใส่สินค้าใหม่ ระบบจะสร้างตะกร้าใบใหม่ให้
ปุ่มตะกร้าบนเมนูไม่อัปเดตจำนวน	ยังไม่ได้รีเฟรช store	ปกติจะอัปเดตเอง ถ้าไม่ขึ้นให้รีเฟรชหน้า (F5)
server ไม่ขึ้น: SECRET_KEY is missing	ไม่มี SECRET_KEY ใน .env	เพิ่มค่าตามหัวข้อ 4
Console: blocked by CORS policy	origin ของหน้าเว็บไม่อยู่ใน whitelist	เพิ่ม origin นั้นใน <code>cors({ origin: [...] })</code> ของ <code>server.js</code>
<code>req.cookies</code> เป็น undefined	เรียก <code>cookieParser()</code> หลัง router	ย้าย <code>app.use(cookieParser())</code> ไปก่อน router

13 สิ่งต่างจากเอกสารแลป

เอกสารแลป	โปรเจกต์นี้	เหตุผล
bcrypt	bcryptjs	มีในโปรเจกต์แล้ว API เหมือนกัน ไม่ต้องคอมไพล์บน Windows
ไฟล์ index.js , routes/ , services/database.js	server.js , src/router/ , database.js	โครงสร้างเดิมของโปรเจกต์
ตัวแปร DBSERVER , DBUSER , ...	DB_HOST , DB_USER , ...	ชื่อที่ database.js ใช้อยู่
postMember ส่ง bodyData กลับทั้งก่อน	ส่งกลับแค่อีเมล ชื่อ วันที่ และผล	ไม่ให้รหัสผ่านถูกส่งกลับไปใน response
SECRET_KEY='thailandgogo'	ค่าสุ่ม 64 ตัวอักษร และ server ไม่ยอมเปิดถ้าไม่มี	เดา key ไม่ได้ = ปลอม token ไม่ได้
แต่ละหน้าเรียก /members/detail เอง และ MainMenu ใช้ watch	authStore.getMember() ที่เดียว + router guard	ทุกหน้าเห็นข้อมูลชุดเดียวกัน ไม่ยิง API ซ้ำ
ตะกร้าส่ง memEmail / cusId มาใน body	อ่านจาก token ใน cookie	ค่าใน body ปลอมเป็นคนอื่นได้ทั้งการดูและการแก้ตะกร้า
postCartDtl รับราคาจาก req.body.pdPrice	อ่านราคาจากตาราง products	ถ้าเชื่อบริษัทจากหน้าเว็บ ผู้ใช้แก้เป็น 0.01 บาทแล้วสั่งซื้อได้
ไม่ได้ตรวจสอบสถานะตะกร้าตอนแก้ไข	ตะกร้าที่ยืนยันแล้วแก้ไม่ได้ (canEditCart)	คำสั่งซื้อที่ยืนยันแล้วไม่ควรเปลี่ยนยอดได้อีก
ปุ่มลบตะกร้า / ยืนยันสั่งซื้อสินค้า มีแต่ confirm()	ต่อกับ delCart และ cfCart จริง	ให้ปุ่มทำงานได้จริง
ตะกร้าเก็บใน localStorage (ของเดิมในโปรเจกต์)	เก็บในฐานข้อมูลตาม Lab 11	ตะกร้าผูกกับสมาชิก ใช้ข้ามเครื่องได้ และมีประวัติคำสั่งซื้อ
ฟอร์ม Bootstrap ตามแลป	ดีไซน์ใหม่ 2 ธีม + 2 ภาษา	ส่วนที่ออกแบบเพิ่มในโปรเจกต์